

1 Adversarial Models in Cryptographic Systems

The security of a cryptographic system is defined with respect to the behavior and capabilities of the entities involved in the system. An entity may be trusted, semi-honest, or malicious depending on the assumptions made about its behavior.

An *adversarial model* specifies which entities may be compromised and what an adversary can cause a compromised entity to do. Thus, the adversarial model is not a property of the functionality itself. Rather, it describes the behavior and capabilities attributed to the entities participating in the system.

1.1 Trusted Entity

A *trusted entity* is an entity that is assumed to behave exactly according to the prescribed protocol and is not compromised by the adversary.

A trusted entity does not intentionally disclose protected information, modify protocol messages, or deviate from the specified computation. Consequently, no adversarial behavior needs to be considered for such an entity.

The assumption that an entity is trusted is the strongest assumption about its behavior and generally results in the simplest protocol design.

1.2 Semi-Honest Entity

A *semi-honest entity*, also called an *honest-but-curious entity*, follows the prescribed protocol correctly but may attempt to learn additional information from everything available to it during the execution.

A semi-honest entity does not modify messages, provide malformed inputs, or deliberately deviate from the protocol. However, it may store and analyze its input, received messages, intermediate values, and other information available to it in an attempt to infer private information.

Thus, security against a semi-honest entity requires that the information available to that entity during an honest execution does not reveal unauthorized information.

1.3 Malicious Entity

A *malicious entity* may arbitrarily deviate from the prescribed protocol. It may provide incorrect inputs, modify or omit messages, send inconsistent information to different entities, or otherwise attempt to violate the correctness, privacy, or other security properties of the protocol.

Therefore, a malicious entity represents a significantly stronger adversarial capability than a semi-honest entity.

A protocol designed to tolerate malicious entities generally requires additional mechanisms to detect or prevent such deviations. Depending on the application, these may include commitments, zero-knowledge proofs, authentication, consistency checks, or verifiable secret sharing.

1.4 Adversarial Model of a System

A cryptographic system may contain many different entities, and each entity can potentially be assigned a different trust assumption. For example, a system may contain users, servers, a dealer, an authority, and communication infrastructure. Each of these entities may be assumed to be trusted, semi-honest, or malicious.

Therefore, defining an adversarial model requires specifying:

1. which entities may be compromised;
2. the behavioral model of each potentially compromised entity;
3. the number of entities that may be simultaneously compromised; and

4. the capabilities available to the adversary through the compromised entities.

For example, in an MPC system with parties $P_1, \dots, P_n$, one may assume that all parties are semi-honest, or that up to t parties may be malicious. These are different adversarial models and generally require different protocol designs.

Thus, *trusted*, *semi-honest*, and *malicious* describe the assumed behavior of individual entities. The adversarial model of the complete system specifies which of these assumptions applies to which entities and how many entities may be under adversarial control.

2 Designing a Cryptographically Secure Scheme

Designing a cryptographic scheme begins with a precise description of the system, its intended functionality, and the adversary against which security is required.

A useful way to organize the design process is to proceed through three fundamental steps:

1. Define the **system model**.
2. Define the **functionality and security goals**.
3. Define the **adversarial model**.

These three components determine what the system is expected to accomplish and what it must remain secure against.

2.1 Step 1: Define the System Model

The first step is to specify the entities involved in the system and their relationships.

We identify the participants, their inputs and outputs, the communication channels between them, and any additional entities such as servers, dealers, authorities, or external services.

For example, in an MPC system, the system model may consist of n parties $P_1, \dots, P_n$, where each party holds a private input and the parties communicate through a network to jointly compute a function.

The system model answers the question:

Who are the entities involved, and how do they interact?

2.2 Step 2: Define the Functionality

Once the system model is specified, we define what the system is supposed to accomplish.

Let the parties provide inputs $x_1, \dots, x_n$. The desired functionality specifies how these inputs should be transformed into outputs. For example, an MPC protocol may be required to compute $y = f(x_1, \dots, x_n)$ while keeping the individual inputs private.

The functionality should specify the legitimate outputs and the information that may be revealed to the participants.

The functionality answers the question:

What should the system compute or accomplish?

2.3 Step 3: Define the Adversarial Model

After specifying the system and its intended functionality, we define the adversarial model.

We determine which entities may be compromised and specify the capabilities available to the adversary. In particular, we decide whether a potentially compromised entity is modeled as trusted, semi-honest, or malicious, and how many entities may be compromised simultaneously.

The adversarial model answers the question:

What can go wrong, and what power does the adversary have over the entities it controls?

2.4 Effect of the Adversarial Model on Protocol Design

The choice of adversarial model directly affects the security requirements and the design of the cryptographic protocol.

For example, if an entity is assumed to be trusted, the protocol does not need to protect against deviations by that entity. If the same entity is modeled as semi-honest, the protocol must prevent it from learning unauthorized information from its protocol view. If it is modeled as malicious, the protocol must additionally tolerate arbitrary deviations by that entity.

Consequently, changing the adversarial model can change the cryptographic mechanisms required by the system. A construction that is secure when entities are assumed to be semi-honest may not remain secure when those entities are allowed to behave maliciously.

Thus, cryptographic protocol design can be viewed as the process of constructing a system that implements the desired functionality while satisfying precisely defined security requirements under a specified adversarial model.

In the following sections, we examine how different adversarial assumptions affect the system model, the security requirements, and ultimately the design of the cryptographic protocol.